

СОФИЙСКИ УНИВЕРСИТЕТ
„СВ. КЛИМЕНТ ОХРИДСКИ“



ФАКУЛТЕТ ПО МАТЕМАТИКА
И ИНФОРМАТИКА

ДЪРЖАВЕН ИЗПИТ

ЗА ПОЛУЧАВАНЕ НА ОКС „БАКАЛАВЪР ПО КОМПЮТЪРНИ НАУКИ“

ЧАСТ I (ПРАКТИЧЕСКИ ЗАДАЧИ)

Драги абсолвенти:

- Попълнете факултетния си номер в горния десен ъгъл на всички листове.
- Пишете само на предоставените листове, без да ги разкопчавате.
- Решението на една задача трябва да бъде на същия лист, на който е и нейното условие (т.е. може да пишете отпред и отзад на листа със задачата, но не и на лист на друга задача).
- Ако имате нужда от допълнителен лист, можете да поискате от квесторите.
- На един лист не може да има едновременно и чернова, и белова.
- Черновите трябва да се маркират, като най-отгоре на листа напишете „ЧЕРНОВА“.
- Ако решението на една задача не се побира на нейния лист, трябва да поискате нов бял лист от квесторите. Той трябва да се защити с телбод към листа със задачата.
- Всеки от допълнителните листове (белова или чернова) трябва да се надпише най-отгоре с вашия факултетен номер.
- Черновите също се предават и се защитават в края на работата.
- Времето за работа по изпита е 3 часа.

Изпитната комисия ви пожелава успешна работа!

Задача 1. Задачата да се реши на езика C++.

- а) Да се попълнят празните места в дадения долу фрагмент от програмен код така, че функцията `hasRepeatingDigits` да проверява рекурсивно дали в дадено цяло число има повтарящи се цифри.

```
bool hasRepeatingDigits(int number, bool digitCount[])
{
    if (number < 0){
        return _____;
    }

    if (number < 10) {
        return _____;
    }

    int digit = number % 10;

    if (_____) {
        return true;
    }

    _____;

    return hasRepeatingDigits(_____, _____);
}

bool hasRepeatingDigits(int number)
{
    bool digitCount[10] = { false };
    return hasRepeatingDigits(number, digitCount);
}
```

- б) Да се реализира функцията `collect` със следния прототип:

```
int* collect(int** matrix, int rows, int cols, int& resultSize);
```

Функцията да връща като резултат едномерен масив, съдържащ онези елементи на целочислената матрица `matrix` с размерност `rows × cols`, в които има повтарящи се цифри. Едномерният масив трябва да бъде заделен динамично с `new`. Размерът му да бъде точно колкото са елементите с повтарящи се цифри и да се връща чрез параметъра `resultSize`. Може да се използва наготово функцията `hasRepeatingDigits`, както и да се реализират допълнителни помощни функции.

- в) Да се реализира програма, която показва работата на функцията, като прочита от стандартния вход размерите на матрицата и елементите ѝ, извиква функцията `collect` и извежда на стандартния изход елементите на масива, който тя връща.

Примерно решение

```
a) bool hasRepeatingDigits(int number, bool digitCount[])
{
    if (number < 0){
        return hasRepeatingDigits(-number, digitCount);
    }

    if (number < 10) {
        return digitCount[number];
    }

    int digit = number % 10;

    if (digitCount[digit]) {
        return true;
    }

    digitCount[digit] = true;

    return hasRepeatingDigits(number / 10, digitCount);
}

bool hasRepeatingDigits(int number)
{
    bool digitCount[10] = { false };
    return hasRepeatingDigits(number, digitCount);
}

б) int* collect(int** matrix, int rows, int cols, int& resultSize)
{
    if (!matrix || rows <= 0 || cols <= 0)
        return nullptr;

    resultSize = 0;
    for (int r = 0; r < rows; ++r){
        for (int c = 0; c < cols; ++c){
            if (hasRepeatingDigits(matrix[r][c]))
                ++resultSize;
        }
    }

    int* result = new(std::nothrow) int[resultSize];
    if (!result) return nullptr;

    resultSize = 0;
    for (int r = 0; r < rows; ++r){
        for (int c = 0; c < cols; ++c){
            if (hasRepeatingDigits(matrix[r][c]))
                result[resultSize++] = matrix[r][c];
        }
    }
    return result;
}
```

в) Използваме функциите от горните подточки.

```
#include <iostream>

bool hasRepeatingDigits(int number);
int* collect(int** matrix, int rows, int cols, int& resultSize);

void clear(int** matrix, int rows)
{
    if (!matrix) return;
    for (int i = 0; i < rows; ++i)
        delete[] matrix[i];
    delete[] matrix;
}

int** createMatrix(int rows, int cols)
{
    int** matrix = nullptr;
    int r = 0;

    try{
        matrix = new int*[rows];
        for (r = 0; r < rows; ++r)
            matrix[r] = new int[cols];
    }catch (std::bad_alloc&){
        clear(matrix, r);
        return nullptr;
    }

    for (r = 0; r < rows; ++r){
        for (int c = 0; c < cols; ++c){
            std::cout << "matrix[" << r << "][" << c << "]=";
            std::cin >> matrix[r][c];
        }
    }
    return matrix;
}

int main()
{
    int rows = 0, cols = 0;
    do {
        std::cout << "Enter size:";
        std::cin >> rows >> cols;
    } while(rows <= 0 || cols <= 0);

    int** matrix = createMatrix(rows, cols);
    if(!matrix) {
        std::cout << "No memory!\n";
        return -1;
    }

    int size = 0;
    int* result = collect(matrix, rows, cols, size);
```

```
    if (result){
        for (int i = 0; i < size; ++i)
            std::cout << result[i] << ' ';
        std::cout << '\n';
        delete[] result;
    } else {
        std::cout << "No memory!\n";
    }

    clear(matrix, rows);
    return 0;
}
```

Критерии за оценяване

- а) За всяко вярно попълнено празно място се присъждат 0,5 точки.
 - б) За пълна и вярна реализация на функцията се присъждат 3 точки. Това включва изискване за коректно заделяне на динамична памет с точния размер.
 - в) За пълна и вярна реализация на главна функция се присъждат 4 точки. Това включва въвеждане на данни с подсигуряване на коректни стойности, заделяне на памет, изпълняване на функцията, извеждане на резултата и освобождаване на заделената памет за матрицата и резултатния масив.
-

Задача 2.

Задачата да се реши на езика C++.

А) Разгледайте дадения фрагмент.

```
1 class base {
2 public:
3     void print() {}
4 };
5
6 class derived : public base {
7     int* data;
8 public:
9     derived() { data = new int;}
10    ~derived() { delete data; }
11    void print() { std::cout << "123";}
12 };
13
14 void run() {
15     base* b = new derived();
16     b->print();
17     delete b;
18 }
```

A.1) Обяснете защо, въпреки че `run()` извиква `print()`, на изхода НЕ СЕ извежда "123".

A.2) Посочете къде в кода има изтичане на памет (memory leak) и обяснете защо (какво се случва).

A.3) Пренапишете кода на класа `base` така, че да НЕ изтича памет и `run()` да извежда на стандартния изход текста "123".

Б) Да се реализира следната йерархия:

Б.1) Абстрактен базов клас `item` с единствен член – публична константна чисто виртуална (pure virtual) функция `value`, която няма аргументи и връща стойност от тип `int`.

Б.2) Клас `num`, който наследява `item` и:

- пази в себе си число `K` от тип `int`;
- има конструктор, който получава число и инициализира с него `K`;
- реализира функцията `value()` да връща числото `K`.

Б.3) Клас `seq`, който наследява `item` и:

- има член-данна `private` динамично заделен с `new` масив от `N` числа от тип `int`;
- има конструктор, който получава число `N`, заделя масива и го запълва с псевдослучайни числа (използвайте `std::rand()`), като ако `N` не е положително число да се хвърля изключение;
- има всички функции от rule of 3 (копиращ конструктор, копиращо присвояване, деструктор) и управлява коректно паметта;
- реализира функцията `value()` да връща първия елемент на масива.

Критерии за оценяване

Сумата от точките се закръгля до цяло число.

А.1) 1 т. за напълно коректен отговор.

А.2) 1 т. за напълно коректен отговор.

А.3) 1 т. за напълно коректен отговор. Съгласно условието, признават се само решения, които коригират base. Не се признават решения, които коригират друг елемент (например функцията run()).

Б.1) 1 точка, ако класът е дефиниран коректно: функцията е публична, чисто виртуална (pure virtual), константна и с правилния тип.

Б.2) 1 точка, ако наследникът е дефиниран коректно и реализира нужната функционалност.

Б.3) По-долу е дадено колко точки носи всеки напълно коректен елемент на решението:

- 1 т. – конструктор, който коректно създава и инициализира масива;
- 1 т. – ако конструкторът от по-горната точка хвърля подходящо изключение ако се подаде некоректна стойност.
- 1 т. – копиращ конструктор;
- 1 т. – копиращо за присвояване;
- 0,5 т. – деструктор (вкл. освобождаване с delete[], а НЕ с delete);
- 0,5 т. – функцията value().

Примерно решение

Забележка: Решението е ориентировъчно. Отговорите може да се формулират по различен начин и да донесат пълни точки, стига да са верни.

А.1) print() НЕ Е виртуална функция. Свързването е статично и затова b->print() е обръщение към base::print(), а не на derived.

А.2) Аналогично, деструкторът на base НЕ Е виртуален. delete b води до обръщение към деструктора на base, а НЕ този на derived.

А.3)

```
class base {
public:
    virtual void print() {}
    virtual ~base() {}
};
```

Б.1)

```
class item {
public:
    virtual int value() const = 0;
};
```

Б.2)

```
class num : public item {
    int K;
public:
    num(int value) : K(value) {}
    virtual int value() const override {
        return K;
    }
};
```

Б.3)

```
class seq : public item {
    int* data;
    int size;
public:
    seq(int N) : size(N) {
        if(N <= 0)
            throw std::invalid_argument(
                "Incorrect size");

        data = new int[size];
        for(int i = 0; i < size; ++i)
            data[i] = std::rand();
    }

    seq(const seq& s) : size(s.size) {
        data = new int[size];
        for(int i = 0; i < size; ++i)
            data[i] = s.data[i];
    }

    seq& operator=(seq const& s) {
        if(this != &s) {
            seq temp(s);
            std::swap(data, s.data);
            std::swap(size, s.size);
        }
        return *this;
    }

    ~seq() { delete [] data; }

    virtual int value() const override {
        return data[0];
    }
};
```

Задача 3. Задачата да се реши на езика C++.

- а) Елементите на едносвързан списък представяме чрез следната структура:

```
struct node {  
    int value;  
    node* next;  
};
```

Празния списък представяме като nullptr.

Да се реализира функция insertSum, която обработва подаден ѝ като аргумент списък, като между всеки два негови последователни елемента с нечетни стойности вмъква нов елемент със стойност – тяхната сума. Паметта за новите елементи да се заделя с new.

Примерен вход:

1 → 2 → 3 → 5 → 3 → 0 → 2

Очакван резултат:

1 → 2 → 3 → 8 → 5 → 8 → 3 → 0 → 2

- б) Да се напише програма, която демонстрира работата на функцията insertSum. Тя трябва да прочете от стандартния вход елементите на свързан списък, върху които да приложи функцията, и след това да изведе на екрана получения резултат. Трябва да се подsigури коректна работа с паметта.
- в) Следната функция трябва да премахва елемент по ключ от двоично дърво за търсене, но в нея са допуснати грешки. Да се посочат кои са грешките и за всяка от тях да се обясни максимално точно защо е грешка и как трябва да се коригира.

```
1 struct node  
2 {  
3     int value;  
4     node* left, *right;  
5 };  
6  
7 void remove(node*& root, int key)  
8 {  
9     if (root->value < key)  
10         return remove(root->right, key);  
11     if (root->value > key)  
12         return remove(root->left, key);  
13     if (!root->left) root = root->right;  
14     else if (!root->right) root = root->left;  
15     else {  
16         node min = root->left;  
17         while (min->left){  
18             min = min->left;  
19         }  
20         root->value = min->value;  
21         min = min->right;  
22     }  
23 }
```


Примерно решение

```
a) void insertSum(node* head)
{
    while(head && head->next){
        if (head->value%2 && head->next->value%2) {
            node* n = new(std::nothrow) node;
            if (!n) break;
            n->value = head->value + head->next->value;
            n->next = head->next;
            head->next = n;
            head = n->next;
        }
        else {
            head = head->next;
        }
    }
}
```

```
б) #include <iostream>
```

```
struct node
{
    int value;
    node* next;
};

void insertSum(node* head);

void clear(node* head)
{
    while (head){
        node* toDel = head;
        head = head->next;
        delete toDel;
    }
}

void print(const node* head)
{
    while (head) {
        std::cout << head->value << " ";
        head = head->next;
    }
    std::cout << "\n";
}

node* build(int cnt)
{
    if (!cnt) return nullptr;

    node* n = new(std::nothrow) node;
    if (n) {
        std::cin >> n->value;
        n->next = build(cnt-1);
    }
}
```

```
    }
    return n;
}

int main()
{
    int cnt;
    do{
        std::cout << "Number of elements: ";
        std::cin >> cnt;
    } while(cnt < 0);
    node* list = build(cnt);

    print(list);
    insertSum(list);
    print(list);
    clear(list);
    return 0;
}
```

- в) – Функцията няма проверка за празно дърво, което включва случая, в който търсеният елемент не се съдържа. Трябва да се добави следният ред преди ред 9: `if (!root) return;`
- Премахнатият елемент не се изтрива и паметта изтича. След ред 12 трябва да се добави следният код: `node* toDel = root;` и след ред 22 да се добави `delete toDel;`
- При два наследника текущият елемент трябва да се замени с минималния вдясно или максималния вляво. Така ред 16 трябва да се замени с `node* min = root->right;`. В същия ред има липсваща звезда.
- Изтриваният елемент трябва да се замени с десния си наследник, за което трябва да се знае неговият адрес. Така типът на `min` е добре да бъде `node**` и съответно всеки достъп до стойността трябва да е през `*min`. Така трябва да се направят следните промени:
- Ред 16 `node** min = &root->right;`
- Ред 17 `while((*min)->left)`
- Ред 18 `min = &((*min)->left);`
- Ред 20 `root->value = (*min)->value;`
- Ред 21 `*min = (*min)->right;`
- Накрая трябва да се промени обектът за изтриване – трябва да се добави следният ред преди ред 21 `toDel = *min;`

Критерии за оценяване

Оценяването е както следва:

- а) За коректна функция се присъждат 3 точки. В това число една точка за вярно обхождане на списъка и достъп до стойностите на два последователни елемента; една точка за коректно създаване и вмъкване на нов елемент и една точка за справяне със случай на липса на памет – не трябва да се допуска теч или невалидни указатели.
- б) За коректна програма се присъждат 3 точки. В това число една точка за коректно построяване на списък; една точка за коректно използване на функцията от подточка а) с извеждане на резултата и една точка за коректно изчистване на паметта накрая.
- в) За всяка от изброените грешки се присъжда по една точка.

Задача 4. ¹ *Задачата да се реши на един от езиците Haskell или Scheme.*

За кандидатстване в училища се държат два изпита, на които се получават две оценки — цяло число точки от 0 до 100. Всяко училище се представя с наредена тройка (списък от три елемента за Scheme) от име (низ), брой свободни места (цяло положително число) и балообразуваща функция, която приема двойка оценки и връща цяло число точки бал. Всяка кандидатура се представя с наредена двойка от списък от имена на училища, подредени по желание на кандидата, и двойка оценки от изпитите. Нека са дадени два непразни списъка `candidates` от кандидатури и `schools` от училища без повторение на имената. Считаме, че всички посочени желания са валидни имена на училища.

Извършването на класиране съобразно бала и желанието на кандидатите е нетривиална задача. По-лесно е да се определи дали един кандидат с **със сигурност** е приет в дадено училище `s`: достатъчно е броят на кандидатите, посочили `s` сред желанията си, чиито бал не е по-нисък от бала на `s` за `s`, да не надвишава броя на свободните места в `s`.

Да се попълнят празните места по-долу, така че списъкът `firstAssured` да съдържа за всеки кандидат съответно името на най-желаното от него училище, в което със сигурност ще бъде приет, а ако такова няма — празен низ. За целта да се реализират следните помощни функции:

- `getSchool`, която по име на училище връща наредената тройка от `schools`, която съответства на училището;
- `assuredAdmission`, която по кандидатура и име на училище `sname` проверява съгласно горния критерий дали приемът на кандидата в училището е гарантиран.

Забележка: Считаме, че `candidates` и `schools` са дефинирани глобално.

Упътване: Позволено е използването на функциите `assv`, `elem`, `filter`, `foldr`, `length`, `map`, `memv`, както и на всички функции от `R5RS` и `Prelude`.

Haskell

```
getSchool name = _____
assuredAdmission _____ sname = _____

where _____ = getSchool sname
firstAssured = _____ (\_____ -> _____) candidates
```

Пример:

```
schools = [("MG", 2, (\(x,y) -> 3 * x + y)), ("LS", 1, (\(x,y) -> 4 * y + x))]
candidates = [(["MG", "LS"], (90, 30)), (["LS"], (60, 85)), (["LS", "MG"], (60, 75))]
firstAssured -> ["MG", "LS", "MG"]
```

Scheme

```
(define (getSchool name) _____)
(define (assuredAdmission candidate sname)
  (let ((school (getSchool sname)))
    _____
    _____
    _____))
(define firstAssured (_____ (lambda (cand)
  _____
  _____))))
```

Пример:

```
(define schools (list (list "MG" 2 (lambda (g) (+ (* 3 (car g)) (cdr g))))
  (list "LS" 1 (lambda (g) (+ (car g) (* 4 (cdr g))))))
(define candidates '(((("MG" "LS") . (90 . 30)) (("LS") . (60 . 85)) (("LS" "MG") . (60 . 75))))
firstAssured -> ("MG" "LS" "MG")
```

¹ *Забележка:* тук са отстранени две технически грешки, които бяха забелязани след разпечатване на изпитната тема:

(1) името на третата дефиниция на Haskell по-долу трябва да е `firstAssured` вместо `firstAdmission` и
(2) вторият параметър във функцията `assuredAdmission` на Scheme трябва да е `sname` вместо `same`.

Примерно решение

Haskell

```
getSchool name = head [ school | school@(sname, _, _) <- schools, name == sname]

assuredAdmission (_, gr) sname = length (filter (\(snames, cgr) -> elem sname snames &&
                                                grading cgr >= grading gr) candidates) <= open
  where (_, open, grading) = getSchool sname

firstAssured = map (\cand@(ns, _) -> head (filter (assuredAdmission cand) ns ++ [""])) candidates
```

Scheme

```
(define (getSchool name) (assv name schools))

(define (assuredAdmission candidate sname)
  (let ((school (getSchool sname)))
    (<= (length (filter (lambda (c) (and (memv sname (car c))
                                         (>= ((caddr school) (cdr c)) ((caddr school) (cdr candidate))))
              candidates)) (cadr school))))

(define firstAssured
  (map (lambda (cand)
        (let ((assured (filter (lambda (sname) (assuredAdmission cand sname)) (car cand))))
          (if (null? assured) "" (car assured)))) candidates))
```

Критерии за оценяване

Ако е писано и по двата езика, се взима по-високият резултат. Сумата от точките се закръгля до цяло число.

Haskell

По ред на празните слотове:

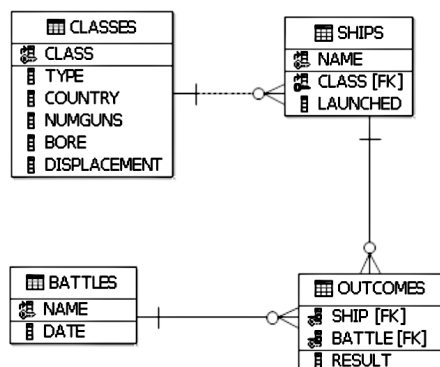
- 2 т. за напълно коректен израз, който намира елемента от списъка `schools`, чиито първи компонент съвпада с параметъра `name`;
 - не се отнемат точки за липса на проверка дали такъв елемент съществува или дали повече от един такъв елемент съществува, понеже по допускане винаги има такъв единствен;
 - ако се връща списък с единствен елемент търсения вместо само елемента, се отнемат **0,5 т.**
- не се добавят или отнемат точки, служи за указване на името на първия параметър на функцията `assuredAdmission` и евентуално напасване с образец по компоненти;
- (оценява се заедно със следващия слот) 4 т. за напълно коректен израз, записан в този и следващия слот, който проверява дали втория компонент на параметъра указан в слот 5 и означаващ училището с име `sname`, е по-голям или равен на дължината на списъка от всички кандидатури, включително и дадената, които отговарят едновременно на следните две условия:
 - параметърът `sname` се среща сред първия компонент на кандидатурата, указващ имената на желаните от кандидата училища;
 - балообразуващата функция, която е последният компонент на параметъра указан в слот 5 и означаващ училището с име `sname`, приложена над втория компонент на кандидатурата, означаващ двойката оценки на кандидата, е по-голям или равен на същата балообразуваща функция, приложена на втория компонент на кандидатурата, подадена като параметър в слот 3.
- оценява се заедно с предишния слот;
- не се добавят или отнемат точки, служи за означаване на върнатия от `getSchool` резултат и евентуално напасване с образец по компоненти;
- 1 т. за функцията `map`;
 - присъжда се 1 т. и за композиция на две коректни извиквания на `map`, като външното проектира по подходящ начин резултата върнат от второто извикване за коректно получаване на търсения резултат;
- не се добавят или отнемат точки, служи за означаване на параметъра на анонимната функция и евентуално напасване с образец по компоненти;
- 3 т. за коректен израз, който за кандидатурата, означена като параметър в слот 7, извлича първия компонент на тази кандидатура, означаващ списъка на имената желаните от кандидата училища, и връща първото от тези имена, за които извикването на функцията `assuredAdmission` с параметри кандидатурата и това име връща истина, а ако такова няма – връща празен низ;
 - отнема се 1 т. за пропусната проверка за случая, в който няма желано от кандидата училище, в което кандидатът е приет, и връщане на празен низ в този случай.

Scheme

По ред на празните слотове:

- 2 т. за напълно коректен израз, който намира елемента от списъка `schools`, чиито първи компонент съвпада с параметъра `name`;
 - не се отнемат точки за липса на проверка дали такъв елемент съществува или дали повече от един такъв елемент съществува, понеже по допускане винаги има такъв единствен;
 - ако се връща списък с единствен елемент търсения вместо само елемента, се отнемат **0,5 т.**
- (оценява се заедно със следващите два слота) 4 т. за напълно коректен израз, записан в този и следващите два слота, който проверява дали втория компонент на `school`, е по-голям или равен на дължината на списъка от всички кандидатури, включително и `candidate`, които отговарят едновременно на следните две условия:
 - параметърът `name` се среща сред първия компонент на параметъра `candidate`, указващ имената на желаните от кандидата училища;
 - балообразуващата функция, която е последният компонент на `school`, приложена над втория компонент на кандидатурата, означаващ двойката оценки на кандидата, е по-голям или равен на същата балообразуваща функция, приложена на втория компонент на параметъра `candidate`.
- оценява се заедно с предишния и следващия слот;
- оценява се заедно с предишните два слота;
- 1 т. за функцията `map`;
 - присъжда се 1 т. и за композиция на две коректни извиквания на `map`, като външното проектира по подходящ начин резултата върнат от второто извикване за коректно получаване на търсения резултат;
- (оценява се със следващия слот) 3 т. за коректен израз, записан в този и следващия слот, който извлича първия компонент на параметъра `sand`, означаващ списъка на имената желаните от кандидата училища, и връща първото от тези имена, за които извикването на функцията `assuredAdmission` с параметри `sand` и това име връща истина, а ако такова няма – връща празен низ;
 - отнема се 1 т. за пропуснатата проверка за случая, в който няма желано от кандидата училище, в което кандидатът е приет, и връщане на празен низ в този случай;
 - Забележка: поради неволен технически пропуск, в дефиницията на функцията `firstAssured` не е подаден списъкът `candidates` като втори аргумент на функцията от по-висок ред, която се очаква да се запише в слот 5. Признават се за коректни решения, които допускат, че този параметър е подаден, както и такива, които го добавят изрично.*
- оценява се заедно с предишния слот.

Задача 5. Дадена е базата от данни Ships, в която се съхранява информация за кораби (Ships) и тяхното участие в битки (Battles) по време на Втората световна война. Всеки кораб е построен по определен стереотип, определящ класа на кораба (Classes).



Таблицата Classes съдържа информация за класовете кораби:

- class — име на класа, първичен ключ;
- type — тип: 'bb' за бойни кораби и 'bc' за бойни крайцери;
- country — държавата, която строи такива кораби;
- numGuns — броят на основните оръдия;
- bore — калибърът им (диаметърът на отвора на оръдието в инчове);

• displacement — водоизместимост в тонове.
Таблицата Ships съдържа информация за корабите:

- name — име на кораб, първичен ключ;
- class — име на неговия клас;
- launched — годината, в която корабът е пуснат на вода.

Таблицата Battles съхранява информация за битките:

- name — име на битката, първичен ключ;
- date — дата на провеждане.

Таблицата Outcomes съдържа информация за резултата от участието на даден кораб в дадена битка, като колоните ship и battle заедно формират първичния ключ:

- ship — име на кораба;
- battle — име на битката;
- result — резултат от битката: 'sunk' за потънал, 'damaged' за повреден и 'ok' за победил.

1) Да се напише заявка, която извежда имената на битките и броя кораби, участвали във всяка битка.

2) Да се попълнят празните места в следната заявка така, че да се изведе името на класа и общата водоизместимост на всички кораби от този клас, които са пуснати на вода преди 1944 г.:

```
SELECT T1.class, _____ (T1.displacement) AS TotalDisplacement
FROM Classes AS T1
JOIN Ships AS T2 ON T1.class = T2.class
WHERE T2._____ < 1944
GROUP BY T1.class;
```

Примерно решение на подзадача 1

```
SELECT battle, COUNT(*) AS NumberOfShipsInBattle  
FROM Outcomes  
GROUP BY battle;
```

Примерно решение на подзадача 2

По ред на празните слотове:

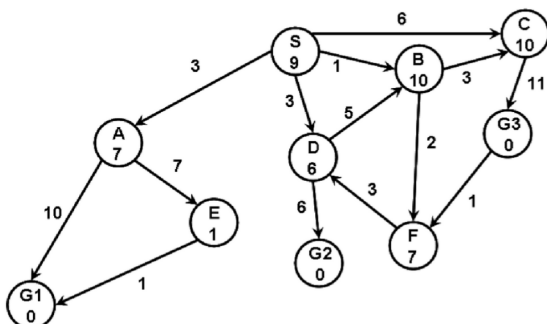
- a) SUM
- б) launched

Критерии за оценяване

- 1) Общо 5 т., като за всяка сгрешена клауза броят на точките се намалява с 2 до достигане на 0 т.
- 2) Общо 5 т., по 2,5 т. на коректно посочен верен отговор.

Задача 6. Оградете *единствения* отговор, който най-добре отговаря на въпроса.

- 1) Кой от следните алгоритми за търсене намира *оптималното* решение?
 - а) търсене в ширина
 - б) изкачване на хълм
 - в) търсене в дълбочина
 - г) алчно търсене
- 2) Алгоритъмът за търсене е *пълен*, ако той:
 - а) винаги намира оптималното решение
 - б) винаги намира решение, ако има такова
 - в) намира всички възможни решения
 - г) обхожда всички възли и намира решение
- 3) Коя от следните техники използва случайността, за да избегне попадане в локални максимуми?
 - а) най-доброто първо
 - б) локално търсене в лъч
 - в) симулирано отгряване/втвърдяване
 - г) градиентно спускане
- 4) Кой от следните проблеми възниква при изкачване на хълм?
 - а) отвесна скала
 - б) плато
 - в) долина
 - г) стръмно спускане
- 5) Алчното търсене използва евристична функция, за да:
 - а) разшири възела, който изглежда най-близо до целта
 - б) разшири възела, който е най-близо до целта
 - в) разшири възела, който е най-скъп
 - г) разшири най-далечния възел
- 6) Каква е разликата между частите от пътя, свързани с функцията на разходите и евристичната функция, на функцията за оценка A^* ?
 - а) Функцията за разходите връща действителната цена от текущия възел до целта, докато евристичната функция връща очакваната цена от текущия възел до целта.
 - б) Функцията за разходите връща прогнозната цена от началния възел до текущия възел, докато евристичната функция връща прогнозната цена от текущия възел до целта.
 - в) Функцията за разходите връща очакваната цена от текущия възел до целта, докато евристичната функция връща действителната цена от началния възел до текущия възел.
 - г) Функцията за разходите връща действителната цена от началния възел до текущия възел, докато евристичната функция връща очакваната цена от текущия възел до целта.
- 7) В дадения по-долу граф за търсене S е началният възел, а $G1$, $G2$ и $G3$ са целеви състояния. Дъгите са обозначени с цената на преминаването им, а евристичната оценка за постигане на цел е показана вътре във възлите. Обхождането на графа се извършва от ляво надясно. За всяка от трите стратегии за търсене по-долу да се посочи кое от целевите състояния е достигнато.



- а) При търсене в ширина целта е достигната в:
- б) При търсене с равномерна цена целта е достигната в:
- в) При търсене с A^* целта е достигната в:
- г) При лакомо търсене целта е достигната в:

Примерно решение

Верните отговори са:

- 1) а)
- 2) б)
- 3) в)
- 4) б)
- 5) а)
- 6) г)
- 7) а) При търсене в ширина целта е достигната в: $G1$
 б) При търсене с равномерна цена целта е достигната в: $G2$
 в) При търсене с A^* целта е достигната в: $G2$
 г) При лакомо търсене целта е достигната в: $G2$

Критерии за оценяване

За всеки верен отговор на въпроси 1 до 6 се присъжда по една точка. За грешен отговор или липса на отговор не се отнемат точки. За вярно посочена цел на всяка от подточките на въпрос 7 се присъжда по една точка.

Задача 7. Нека фиксираме азбуката $\Sigma = \{a, b\}$. За произволен език $L \subseteq \Sigma^*$ дефинираме $\text{коп}(L) = \{ww : w \in L\}$.

- (1) Да се намери безкраен език $L_1 \subseteq \Sigma^*$, за който $\Sigma^* \setminus L_1$ също е безкраен, със свойството, че $\text{коп}(L_1)$ е регулярен.
- (2) Да се намери регулярен език L_2 , такъв че $\text{коп}(L_2)$ не е регулярен.
- (3) Да се докаже, че $\text{коп}(L_2)$ не е регулярен.

Примерно решение и критерии за оценяване

- (1) Да положим $L_1 = (aa)^*$. Тъй като думите от вида a^{2n} са от L_1 , то L_1 е безкраен. Тъй като думите от вида a^{2n+1} са от езика $\Sigma^* \setminus L_1$, то $\Sigma^* \setminus L_1$ също е безкраен. Освен това

$$\begin{aligned}\text{коп}(L_1) &= \{ww : w \in (aa)^*\} \\ &= \{a^{2n}a^{2n} : n \in \mathbb{N}\} \\ &= \{a^{4n} : n \in \mathbb{N}\} \\ &= (aaaa)^*\end{aligned}$$

Следователно $\text{коп}(L_1)$ е регулярен. **(2 точки)**

- (2) Нека положим $L_2 = a^*b$. Очевидно е, че L_2 е регулярен език, тъй като може да бъде представен чрез регулярен израз. Ще докажем, че $\text{коп}(L_2)$ не е регулярен. **(2 точки)**
- (3) За езика L_2 имаме следното:

$$\begin{aligned}\text{коп}(L_2) &= \{ww : w \in a^*b\} \\ &= \{a^nba^nb : n \in \mathbb{N}\}\end{aligned}$$

За да докажем, че $\text{коп}(L_2)$ не е регулярен, ще приложим критерия за проверка за нерегулярност на език, който имаме като следствие от лемата за разрастване (покачване) за регулярни езици.

(v) Разглеждаме произволно естествено число $p \geq 1$.

(э) Избираме дума $s = a^pba^pb \in \text{коп}(L_2)$. Имаме свойството, че $|s| = p + 1 + p + 1 \geq p$. **(1 точка)**

(v) Разглеждаме произволно разбиване на думата s на три части като $s = xyz$ със свойствата $|xy| \leq p$ и $|y| \geq 1$. Тъй като $|xy| \leq p$, то xy може да включва само символи a от началото на думата a^pba^pb . Понеже $|y| \geq 1$, то $y = a^k$ за някое $k \geq 1$ и обхваща букви a от първата половина на a^pba^pb . **(2 точки)**

(э) Остава да посочим естествено число i , за което $xy^iz \notin \text{коп}(L_2)$. Нека разгледаме думата $xy^0z = xz = a^{p-k}ba^pb$. Остава да проверим, че $xy^0z \notin \text{коп}(L_2)$. За да бъде $xz \in \text{коп}(L_2)$, трябва $xz = ww$ за някоя дума $w \in L_2$. Това означава, че $xz = a^mba^mb$ за някое $m \geq 0$. Следователно $a^{p-k}ba^pb = a^mba^mb$, откъдето $p - k = m$ и $p = m$. Това означава, че $p - k = p$, което е противоречие, защото $k > 0$. Заклучаваме, че $xy^0z \notin \text{коп}(L_2)$. **(3 точки)**

Задача 8. Да се пресметне интегралът:

$$\int_0^1 x^5 \ln(x^4 + 1) dx$$

Примерно решение

Преобразуваме последователно:

$$\begin{aligned}
 \int_0^1 x^5 \ln(x^4 + 1) dx &= \\
 (1 \text{ т.}) \quad &= \frac{1}{6} \int_0^1 \ln(x^4 + 1) d(x^6) = \\
 (1 \text{ т.}) \quad &\quad \quad \quad (\text{полагаме } t = x^2) \quad \quad \quad = \frac{1}{6} \int_0^1 \ln(t^2 + 1) d(t^3) = \\
 (1 \text{ т.}) \quad &\quad \quad \quad (\text{интегрираме по части}) \quad \quad \quad = \frac{1}{6} [\ln(t^2 + 1)t^3]_0^1 - \frac{1}{6} \int_0^1 t^3 d[\ln(t^2 + 1)] = \\
 (1 \text{ т.} + 1 \text{ т.}) \quad &\quad \quad \quad = \frac{1}{6} [\ln(2) - 0] - \frac{1}{6} \int_0^1 t^3 \frac{2t}{t^2 + 1} dt = \\
 (1 \text{ т.}) \quad &\quad \quad \quad = \frac{1}{6} \ln(2) - \frac{1}{3} \int_0^1 \frac{t^4 + t^2 - t^2 - 1 + 1}{t^2 + 1} dt = \\
 (1 \text{ т.}) \quad &\quad \quad \quad = \frac{1}{6} \ln(2) - \frac{1}{3} \int_0^1 \left[t^2 - 1 + \frac{1}{t^2 + 1} \right] dt = \\
 (1 \text{ т.}) \quad &\quad \quad \quad = \frac{1}{6} \ln(2) - \frac{1}{3} \left[\frac{1}{3} t^3 - t + \arctg(t) \right]_0^1 = \\
 (1 \text{ т.}) \quad &\quad \quad \quad = \frac{1}{6} \ln(2) - \frac{1}{9} + \frac{1}{3} - \frac{1}{3} \cdot \frac{\pi}{4} = \\
 (1 \text{ т.}) \quad &\quad \quad \quad = \frac{1}{6} \ln(2) + \frac{2}{9} - \frac{\pi}{12}
 \end{aligned}$$

Критерии за оценяване

За всяка вярна стъпка от пресмятането се присъжда точка, съгласно посочените в примерното решение.

Чернова